

Progetto 6: Valutazione e Fine-Tuning di Modelli Linguistici (LLM)

Gruppo di Progetto

4 marzo 2026

1 Introduction

1.1 Problem Description

Il **Progetto 6: Valutazione e Fine-Tuning di Modelli Linguistici (LLM)** ha come obiettivo principale l'analisi comparativa di modelli linguistici pre-addestrati, valutandone le capacità di adattamento a task specifici. Il focus del lavoro risiede nell'esecuzione pratica di diverse strategie di **Fine-Tuning** su modelli distinti, al fine di misurarne quantitativamente le prestazioni post-adattamento e farne emergere, tramite confronto diretto, i rispettivi **punti di forza e di debolezza**.

Nello specifico, il nostro gruppo ha selezionato il task di **Emotion Recognition** (Classificazione delle Emozioni) applicato a testi brevi provenienti dai social media. La rilevanza di questo problema è elevata nell'attuale panorama digitale: la capacità di analizzare automaticamente il "sentiment" e le sfumature emotive (gioia, tristezza, rabbia, ecc.) su piattaforme come Twitter è cruciale per applicazioni di monitoraggio della brand reputation, analisi sociale e customer care automatizzato.

Il pubblico di riferimento include ricercatori NLP, data scientist e sviluppatori di assistenti virtuali empatici. Una soluzione efficace permette di automatizzare l'analisi di grandi moli di dati non strutturati (Big Data) con costi ridotti rispetto all'annotazione manuale.

1.2 Proposed Solution

Per affrontare il problema, abbiamo adottato un **approccio sperimentale comparativo**, testando diverse tecniche di fine-tuning su una selezione eterogenea di modelli pre-addestrati. L'obiettivo centrale è stato confrontare l'efficacia delle diverse strategie di adattamento in relazione alle architetture sottostanti.

La nostra soluzione si articola nei seguenti punti:

- **Modelli Selezionati:** Abbiamo condotto esperimenti su quattro architetture distinte per valutare l'impatto della dimensione e della specializzazione del pre-training:
 - `vinai/bertweet-base`: Modello domain-specific per Twitter.
 - `distilbert/distilbert-base-uncased`: Modello distillato (leggero) per testare l'efficienza.
 - `google-bert/bert-base-uncased`: La baseline standard per il NLP.

– FacebookAI/roberta-base: Versione ottimizzata di BERT.

- **Approccio Metodologico:** Un confronto sistematico tra diverse metodologie di apprendimento:
 1. **Transfer Learning (Frozen):** Addestramento del solo classificatore mantenendo il corpo del modello congelato.
 2. **Complete Fine-Tuning:** Aggiornamento di tutti i pesi del modello per l'adattamento al task. Si specifica inoltre che, su alcuni modelli, si è scelto di utilizzare anche **LoRA (Low-Rank Adaptation)**, una tecnica PEFT che inietta matrici addestrabili nei layer del modello.
 3. **In-Context Learning (Zero/Few-Shot):** Per testare le capacità native senza alcun aggiornamento dei pesi.
- **Sfide Computazionali:** Le principali difficoltà affrontate sono state la gestione della **Context Window** limitata (tipica dei modelli Encoder-Only), che ha imposto vincoli severi sulle strategie di Few-Shot Learning, e la sfida di massimizzare le performance di generalizzazione nonostante il **basso numero di parametri** dei modelli “base” e “distilled” selezionati, richiedendo un’ottimizzazione fine degli iperparametri. Un’ulteriore sfida è rappresentata dallo **sbilanciamento delle classi** nel dataset: come evidenziato in Figura 1, le classi *Joy* e *Sadness* dominano il training set (rispettivamente ~ 5300 e ~ 4700 campioni), mentre *Surprise* conta appena ~ 600 esempi, rendendo più difficile la generalizzazione sulle emozioni minoritarie.

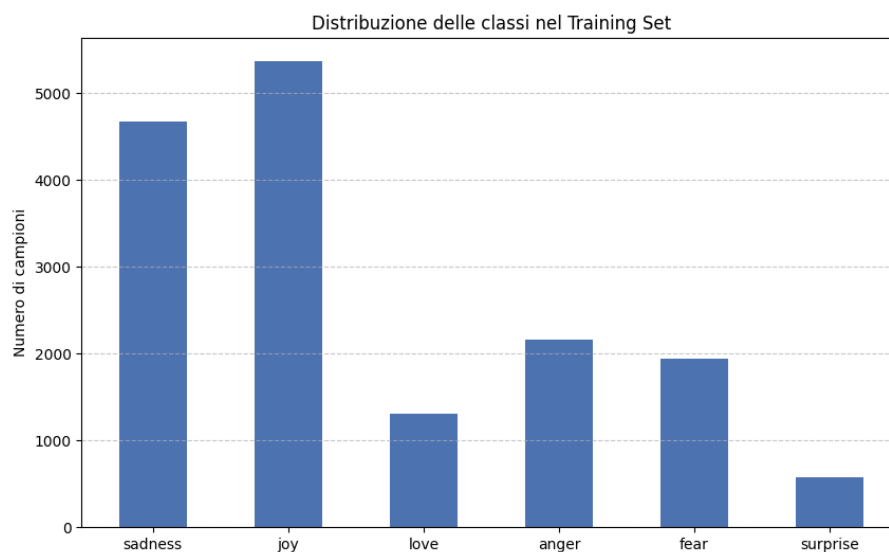


Figura 1: Distribuzione delle classi nel Training Set: evidente sbilanciamento tra *Joy/Sadness* e *Surprise*.

Task Distribution: Il lavoro è stato parallelizzato assegnando a ciascun membro del gruppo la responsabilità completa di una specifica architettura. Ogni componente ha curato l’analisi esplorativa, il pre-processing, il fine-tuning e la valutazione del proprio modello, per poi convergere in una fase finale di confronto dei risultati.

- **[Membro 1]:** Responsabile per `vinai/bertweet-base`.

- **[Membro 2]:** Responsabile per `distilbert/distilbert-base-uncased`.
- **[Membro 3]:** Responsabile per `google-bert/bert-base-uncased`.
- **[Membro 4]:** Responsabile per `FacebookAI/roberta-base`.

Summary of Achieved Results: TODO

1.3 [extra] Literature Review

In letteratura, il benchmark per il riconoscimento delle emozioni su testo fa spesso riferimento al dataset `dair-ai/emotion`, introdotto originariamente da Saravia et al. (2018)[1] nel lavoro “CARER: Contextualized Affect Representations for Emotion Recognition”. Questo dataset è stato costruito utilizzando hashtag come etichette deboli (distant supervision) per classificare i tweet in categorie emotive fondamentali.

Per quanto riguarda i modelli, lo stato dell’arte su dati provenienti da Twitter è rappresentato da architetture domain-specific come **BERTweet**, presentato da Nguyen et al. (2020) [2]. **BERTweet** utilizza la stessa architettura di BERT-base ma è stato pre-addestrato su un corpus massivo di 850 milioni di tweet inglesi, dimostrando prestazioni superiori rispetto a RoBERTa e XLM-R su task di classificazione testuale, POS tagging e NER in ambito social media.

Tuttavia, approcci di Full Fine-Tuning su questi modelli richiedono risorse computazionali significative. Per mitigare questo problema, Hu et al. (2021) [3] hanno proposto **LoRA (Low-Rank Adaptation)**. Questa tecnica congela i pesi del modello pre-addestrato e inietta matrici di decomposizione di rango ridotto nei layer del Transformer, riducendo drasticamente il numero di parametri addestrabili (fino a 10.000 volte in meno rispetto al fine-tuning completo su GPT-3) senza introdurre latenza in inferenza. Il nostro lavoro dimostra come l’applicazione di LoRA su BERTweet, se ben calibrata, possa eguagliare i metodi tradizionali a una frazione del costo computazionale.

2 Proposed Method

2.1 Solution Choice

Abbiamo scartato l’uso di LLM generativi generalisti (es. GPT-4, Llama-3) per motivi di latenza in inferenza e costi computazionali, preferendo architetture **Encoder-Only** (come **BERTweet** e le varianti di BERT/RoBERTa). I modelli Encoder sono strutturalmente superiori nei task di classificazione pura grazie all’attenzione bidirezionale che cattura l’intero contesto della frase simultaneamente.

Per quanto riguarda le tecniche di addestramento e adattamento, abbiamo optato per un confronto a tutto tondo al fine di comprendere appieno i trade-off tra risorse impiegate e prestazioni ottenute. Inizialmente abbiamo valutato il **Transfer Learning** classico (approccio *Feature Extraction*), che pur essendo computazionalmente leggero e veloce presenta lo svantaggio di congelare il corpo del modello, limitandone drasticamente la capacità di apprendimento su domini complessi. Successivamente, per stabilire un limite superiore (*upper bound*) alle performance, abbiamo testato il **Complete Fine-Tuning**, il quale aggiorna la totalità dei pesi garantendo un’ottima accuratezza, ma risultando al contempo lento e molto oneroso in termini di memoria. Per avere un quadro sperimentale completo, abbiamo esplorato anche l’**In-Context Learning** per testare le capacità

native (*zero-shot* e *few-shot*) dei modelli senza effettuare alcun aggiornamento dei pesi, scontrandoci tuttavia con i severi limiti imposti dalla *context window* ristretta tipica di queste architetture.

Infine, proprio per superare l'inefficienza e i costi del fine-tuning completo e la rigidità del transfer learning classico, su alcuni modelli si è scelto di implementare **LoRA (Low-Rank Adaptation)**. Questa tecnica permette di adattare efficacemente le rappresentazioni interne del modello al task specifico delle emozioni iniettando e modificando solo poche matrici di peso aggiuntive.

2.2 Methodology for performance measurement

Le prestazioni sono state valutate utilizzando un set di metriche standard per la classificazione multiclasse, selezionate per fornire una visione sia quantitativa che qualitativa:

- **Accuracy:** Rappresenta la frazione di predizioni corrette rispetto al totale dei campioni. Sebbene sia la metrica più intuitiva per una valutazione globale, in questo progetto viene utilizzata solo come riferimento di base, poiché può risultare ingannevole in presenza di dataset sbilanciati (un modello che predice sempre la classe maggioritaria avrebbe comunque un'accuratezza elevata).
- **Macro F1-Score:** È la media armonica tra *Precision* e *Recall*, calcolata indipendentemente per ogni classe e poi mediata (senza pesare per la numerosità delle istanze). Questa è la metrica **critica** per il nostro task a causa del forte sbilanciamento del dataset (le classi *Joy* e *Sadness* sono molto più frequenti di *Surprise* e *Love*). L'uso del *Macro-average* penalizza i modelli che ignorano le classi minoritarie, garantendo che ogni emozione abbia lo stesso peso nella valutazione finale.

3 Experimental Results

3.1 bertweet-base

3.1.1 Demonstration and Technologies

Per garantire la riproducibilità degli esperimenti, il progetto è stato sviluppato utilizzando le seguenti tecnologie e versioni:

- **Environment:** Ambiente di sviluppo locale (Python 3.13.12).
- **Librerie:**
 - `transformers` (HuggingFace) per la gestione del modello BERTweet.
 - `peft` (HuggingFace) per l'implementazione di LoRA.
 - `datasets` e `evaluate` per la gestione dei dati e delle metriche.
 - `scikit-learn` per il calcolo delle matrici di confusione.
- **Istruzioni Demo:** Il notebook fornito esegue automaticamente il download del dataset, il caricamento dei pesi LoRA ottimizzati e la valutazione sul validation set, stampando sul notebook le metriche finali.

3.1.2 Results

La configurazione migliore identificata (**Full Fine Tuning**) ha ottenuto i seguenti risultati sul Test Set:

- **Accuracy:** 93.40%
- **Macro F1-Score:** 89.90%
- **Training Time:** ~ 31 minuti (10 epoche).

Questa configurazione utilizzava un Learning Rate di $3e^{-5}$ e un Batch Size di 8, con tutti i parametri del modello aggiornati durante il training (*full fine tuning*) senza alcun freezing dei layer.

Nota: LoRA ha ottenuto risultati comparabili con Accuracy 93.20% e Macro F1-Score 89.10%, rappresentando un'alternativa efficiente in termini di parametri addestrabili a fronte di prestazioni molto simili.

3.1.3 Ablation Study: Analysis of Configurations

Lo studio di ablation è stato condotto in tre fasi distinte per isolare l'impatto dei singoli iperparametri e delle limitazioni architetturali.

3.1.4 1. Impatto del Context Window (In-Context Learning)

Abbiamo valutato l'efficacia dell'aggiunta di esempi (k -shot) nel prompt. I risultati mostrano che aumentare k non migliora le prestazioni a causa della limitata finestra di contesto di BERTweet (128 token). Con $k = 5$, il modello inizia a troncare il prompt, perdendo informazioni cruciali.

Shots (k)	Accuracy	F1-Score	Truncated	Note
0	49.30%	24.63%	0	Baseline Zero-Shot (Migliore).
1	46.87%	22.28%	0	Peggioramento
3	48.40%	24.59%	0	Nessun miglioramento significativo.
5	48.88%	25.77%	25	25 esempi su 2000 troncati.

Tabella 1: Ablation su In-Context Learning

3.1.5 2. Tuning del Transfer Learning (Frozen)

Mantenendo il corpo del modello congelato, abbiamo condotto una *Grid Search* completa combinando tre valori di Learning Rate ($LR \in \{1e^{-3}, 1e^{-4}, 1e^{-5}\}$) con due dimensioni di Batch Size ($BS \in \{8, 16\}$), per un totale di 6 configurazioni sperimentali. I risultati indicano che un LR più aggressivo ($1e^{-3}$) è strettamente necessario per addestrare efficacemente la testa di classificazione da zero; tuttavia, l'accuracy massima rimane bloccata attorno al 54%, confermando l'insufficienza del metodo *Frozen* per questo task.

Learning Rate	Batch Size	Accuracy	F1-Score
$1e^{-3}$	16	54.15%	29.62%
$1e^{-3}$	8	53.60%	29.33%
$1e^{-4}$	8	50.70%	21.86%
$1e^{-4}$	16	50.40%	21.07%
$1e^{-5}$	8	44.55%	17.36%
$1e^{-5}$	16	41.10%	15.10%

Tabella 2: Grid Search completa sul Transfer Learning (Frozen): tutte le 6 configurazioni ordinate per Accuracy decrescente.

3.1.6 3. Tuning del Full Fine Tuning

Per il Full Fine Tuning, abbiamo condotto una *Grid Search* su tre valori di Learning Rate tipici per questo approccio ($LR \in \{2e^{-5}, 3e^{-5}, 5e^{-5}\}$) con due dimensioni di Batch Size ($BS \in \{8, 16\}$), per un totale di 6 configurazioni sperimentali. I risultati mostrano che il modello è molto sensibile al Learning Rate: $LR = 5e^{-5}$ con Batch Size 16 causa un collasso del training (Accuracy 34.75%), mentre le configurazioni con LR più conservativi raggiungono tutte accuracy superiori al 92%. La configurazione ottimale ($LR = 3e^{-5}$, $BS = 8$) ha ottenuto la migliore Accuracy e F1-Score.

Learning Rate	Batch Size	Accuracy	F1-Score
$3e^{-5}$	8	93.55%	89.76%
$2e^{-5}$	8	93.25%	89.43%
$3e^{-5}$	16	93.15%	88.88%
$5e^{-5}$	8	93.10%	89.05%
$2e^{-5}$	16	92.90%	89.10%
$5e^{-5}$	16	34.75%	8.60%

Tabella 3: Grid Search completa sul Full Fine Tuning: tutte le 6 configurazioni ordinate per Accuracy decrescente.

3.1.7 4. Configurazione Ottimale di LoRA

Per individuare l'architettura PEFT ideale, abbiamo eseguito una *Grid Search* testando 24 combinazioni diverse di iperparametri: Rango ($r \in \{8, 16\}$), Moduli Target ($[q, v]$, $[q, k, v]$, $[q, v, k, dense]$), Learning Rate ($2e^{-4}$, $2e^{-5}$) e Batch Size (8, 16). I dati confermano che targettare **tutti i moduli lineari** (`[query, value, key, dense]`) è consistentemente superiore rispetto a limitarsi ai soli layer di attenzione. Il Learning Rate si rivela il fattore più critico: $LR = 2e^{-5}$ causa un degrado significativo delle prestazioni in tutte le configurazioni, mentre $LR = 2e^{-4}$ garantisce convergenza stabile. Il rango $r = 16$ ottiene un margine di miglioramento modesto rispetto a $r = 8$.

Rango	Target Modules	LR	BS	Accuracy	F1-Score
16	[q, v, k, dense]	$2e^{-4}$	8	93.35%	89.71%
16	[q, v, k, dense]	$2e^{-4}$	16	93.25%	89.00%
8	[q, v, k, dense]	$2e^{-4}$	8	93.15%	88.72%
16	[q, k, v]	$2e^{-4}$	8	92.20%	87.52%
16	[q, v]	$2e^{-4}$	8	92.15%	87.32%
8	[q, v]	$2e^{-4}$	8	91.90%	88.06%
16	[q, v, k, dense]	$2e^{-5}$	8	90.70%	85.91%
8	[q, v, k, dense]	$2e^{-5}$	8	89.25%	83.59%
8	[q, v]	$2e^{-5}$	8	79.25%	66.71%

Tabella 4: Selezione rappresentativa dei risultati della Grid Search LoRA, ordinati per Accuracy decrescente.

3.1.8 Sintesi Finale e Confronto

Il grafico seguente riassume il salto prestazionale ottenuto passando attraverso le diverse strategie.

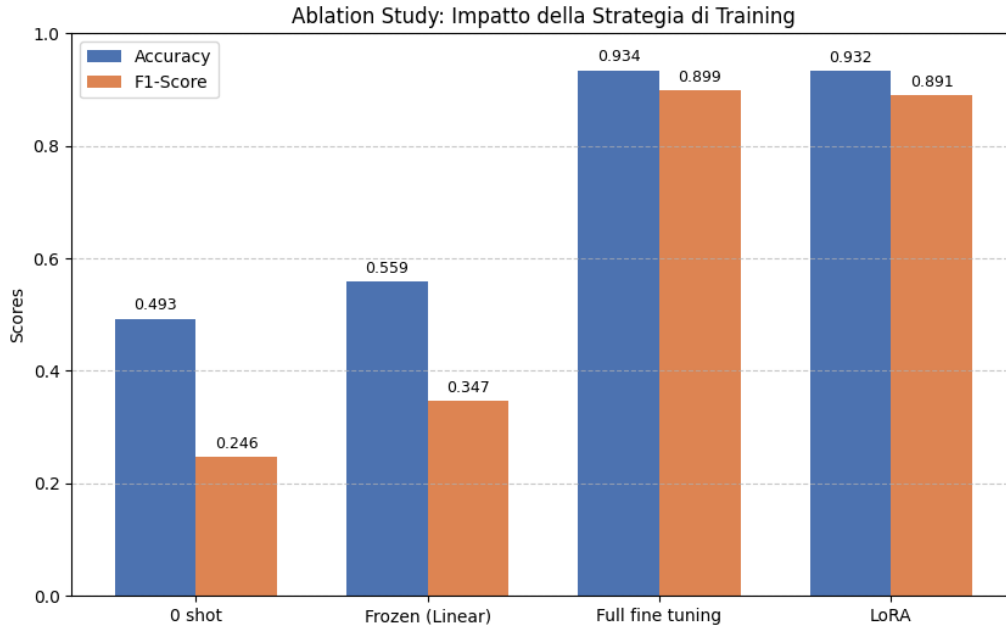


Figura 2: Confronto visivo delle Accuracy tra le strategie testate. Si nota il netto divario tra i metodi Frozen e LoRA.

In conclusione, la tabella riassuntiva delle migliori configurazioni per categoria è la seguente:

Strategia	Accuracy	F1	Conclusioni
In-Context ($k = 0$)	49.30%	24.63%	Inadatto per task emotivi specifici.
Transfer Learning	55.90%	34.70%	Limitato dalle feature pre-addestrate.
LoRA ($r = 16$, all-linear)	93.20%	89.10%	Ottimo rapporto parametri/prestazioni.
Full Fine Tuning	93.40%	89.90%	Best Model. Aggiornamento completo dei pesi con $LR = 3e^{-5}$.

Tabella 5: Riepilogo finale Ablation Study

3.1.9 Results Discussion

I risultati ottenuti confermano che il Full Fine Tuning su un modello pre-addestrato su dominio specifico costituisce un approccio estremamente efficace per task di classificazione emotiva. Raggiungere un’accuratezza del 93.40% aggiornando l’intero insieme di parametri ha permesso al modello di adattarsi profondamente alle caratteristiche del linguaggio informale tipico dei social media. L’elevato Macro F1-Score (89.90%) conferma che il modello non è collassato sulle classi maggioritarie (come *Joy* o *Sadness*), ma ha imparato a discriminare con efficacia anche le emozioni meno frequenti nel dataset.

Tuttavia, l’analisi qualitativa degli errori, supportata dalla Matrice di Confusione (Figura 3), ha rivelato che la maggior parte delle misclassificazioni residue non è casuale, ma si concentra su specifiche coppie di emozioni semanticamente vicine o contestualmente ambigue:

1. **Love vs Joy:** Questa rappresenta la confusione più frequente, con il 10% degli esempi di *Love* classificati erroneamente come *Joy*. È dovuta alla forte sovrapposizione semantica nei testi brevi tipici dei social network, dove parole di affetto, forte apprezzamento ed entusiasmo vengono usate in modo intercambiabile. Frasi che esprimono una profonda gioia spesso contengono termini legati all’amore (es. "I absolutely love this!"), rendendo il confine decisionale estremamente sfumato, a volte anche per un annotatore umano.
2. **Fear vs Surprise:** Il 10% degli esempi di *Surprise* viene classificato erroneamente come *Fear*, e il 4% degli esempi di *Fear* come *Surprise*. Nei tweet, intrinsecamente brevi e spesso privi di contesto esteso, la reazione a un evento scioccante risulta difficile da disambiguare: senza abbastanza indizi testuali, il modello fatica a distinguere se la sorpresa sia di natura neutra/positiva oppure legata a una minaccia.

Le classi meglio classificate sono *Sadness* (98%) e *Joy* (96%), probabilmente grazie alla loro maggiore frequenza nel dataset e alla ricchezza del loro vocabolario caratteristico. *Surprise* risulta la classe più difficile (88%), coerentemente con la sua natura semanticamente ibrida.

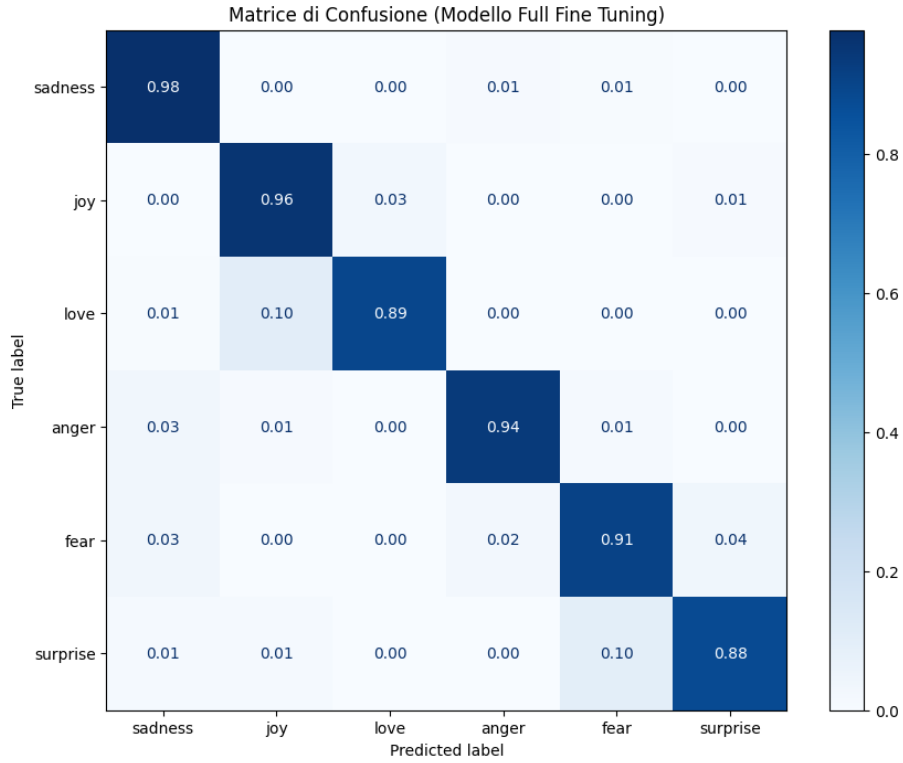


Figura 3: Matrice di Confusione del modello Full Fine Tuning.

3.1.10 Method Validity

Il metodo di Full Fine Tuning si è dimostrato la strategia più efficace, confermando che l'aggiornamento completo dei pesi su un modello pre-addestrato su dominio specifico rappresenta la scelta ottimale quando le risorse computazionali lo consentono. LoRA ha ottenuto risultati comparabili (93.20% di Accuracy) con un numero significativamente inferiore di parametri addestrabili, rappresentando un'alternativa valida in contesti con vincoli di memoria o tempo. Al contrario, lo studio di ablation su **In-Context Learning** ha evidenziato un limite strutturale critico: l'architettura BERTweet, progettata con una finestra di contesto ridotta (128 token), **non è idonea** a strategie Few-Shot ($k > 3$). Gli esempi dimostrativi tendono a saturare la memoria e troncare il prompt, peggiorando le performance invece di migliorarle, rendendo il fine-tuning una scelta obbligata per questo tipo di architetture.

3.2 distilbert-base-uncased

3.3 bert-base-uncased

3.4 roberta-base

3.4.1 Model

RoBERTa sta per **R**obustly **o**ptimized **BERT** approach ed è un modello pre-addestrato con l'obiettivo di Masked Language Modeling (MLM). Data in input una frase, il modello maschera casualmente il 15% delle parole, elaborando l'intera frase mascherata e deve

predire le parole che sono state oscurate. Il modello in questione (RoBERTa-base) si presenta con un numero di parametri allenabili pari a 124,650,246.

3.4.2 Demonstration and Technologies

L'analisi del modello è stata eseguita utilizzando le seguenti tecnologie:

- **Environment:** Google Colab (Python 3) con runtime T4 GPU.
- **Librerie:**
 - `transformers` libreria di HuggingFace per la gestione dei modelli
 - `datasets` e `evaluate` per la gestione dei dati e delle metriche.
 - `scikit-learn` per il calcolo delle matrici di confusione.
 - `peft` libreria di HuggingFace per l'implementazione di LoRA.
 - `numpy` e `matplotlib` per l'analisi dei dati.
 - `google` già presente su Colab per la connessione a Google Drive.

Per poter riprodurre i risultati è necessario rieseguire tutte le celle del notebook. A causa dei limiti di utilizzo della T4 GPU di Google Colab, l'esecuzione potrebbe essere interrotta prima del completamento. Per questo motivo è stato collegato Google Drive per il salvataggio dei modelli.

3.4.3 Results

Dopo aver testato differenti tecniche, l'esito migliore è stato raggiunto con la strategia di **Full Fine-Tuning** che ha ottenuto i seguenti risultati sul Validation Set:

- **Accuracy:** 94.15%
- **Macro F1-Score:** 91.86%
- **Training Time:** ~10:30 (5/10 epoche - Early stopping dopo 2 epoche).

La configurazione utilizza un Learning Rate di $2e^{-5}$, un Batch Size di 16, un Weight Decay di 0.01 e un indice di Warmup Steps di 0.1.

3.4.4 Ablation Study: Analysis of Configurations

Lo studio di Ablation è stato svolto in quattro fasi dell'analisi con lo scopo di determinare la configurazione di iperparametri di maggior efficienza:

- **A. Transfer Learning:** i pesi del modello base vengono "congelati", impedendo qualsiasi aggiornamento durante il training. Solo la classification head viene allenata, adattando il modello RoBERTa alle classi del task.
- **B. Full Fine-Tuning:** tutti i pesi del modello base vengono aggiornati durante il training, consentendo al modello di specializzarsi completamente sul task. Questo approccio però richiede risorse computazionali significative.

- **C. LoRA (Low Rank Adaptation):** anzichè aggiornare direttamente i pesi del modello, si introducono due matrici di rango ridotto. Solo queste matrici vengono poi addestrate, riducendo drasticamente il numero di parametri.
- **D. In-Context Learning (ICL):** il modello viene utilizzato senza alcun aggiornamento dei pesi. Nel caso few-shot, alcuni esempi etichettati vengono forniti direttamente nel prompt guidando la predizione.

A. Transfer Learning (Allineamento solo della Classification Head)

I parametri del corpo del modello base sono stati "congelati" ad eccezione della testa di classificazione (595, 206 parametri allenabili) ed è stata eseguita una ricerca completa comparando sei configurazioni differenti tra tre valori di Learning Rate $\in \{1e^{-3}, 1e^{-4}, 1e^{-5}\}$ e due dimensioni di Batch Size $\in \{16, 32\}$. Analizzando i risultati della ricerca risulta che, con l'utilizzo di un valore di learning rate maggiore ($1e^{-3}$) si ottiene un addestramento della testa di classificazione più efficiente rispetto a valori inferiori:

- $1e^{-3}$ produce le performance migliori in entrambe le configurazioni di batch size, mentre valori inferiori degradano progressivamente.
- In particolare, $1e^{-5}$ produce risultati identici indipendentemente dalla batch size (34.75% accuracy, 8.60% F1), suggerendo che un learning rate eccessivamente basso impedisce qualsiasi aggiornamento significativo dei pesi della testa di classificazione.

I risultati migliori raggiunti risultano avere un valore di accuracy del 57.95% e un valore di F1-Score del 32.50%, non abbastanza efficienti per il task di text emotion classification rispetto ad altre strategie analizzate in seguito.

Learning Rate	Batch Size	Accuracy	F1-Score
$1e^{-3}$	16	57.95%	32.50%
$1e^{-3}$	32	56.80%	30.66%
$1e^{-4}$	16	51.45%	20.68%
$1e^{-4}$	32	49.65%	19.88%
$1e^{-5}$	16	34.75%	08.60%
$1e^{-5}$	32	34.75%	08.60%

Tabella 6: Risultati delle configurazioni per l'allineamento della testa di classificazione

B. Full Fine-Tuning

La seconda tecnica applicata è stata il Full Fine-Tuning. In questo caso è stato allenato il 100% dei parametri (124, 650, 246) ed è stata eseguita una ricerca completa comparando sei configurazioni differenti tra tre valori di Learning Rate $\in \{1e^{-5}, 2e^{-5}, 3e^{-5}\}$ e due dimensioni di Batch Size $\in \{16, 32\}$ come citato nel paper di riferimento presentato da Yinhan Liu et al. (2019) [4]. La configurazione che ha ottenuto i risultati migliori utilizza un valore di Learning Rate di $2e^{-5}$ e una dimensione di Batch Size di 16, raggiungendo un valore di accuracy del 93.10% e un valore di F1 macro del 89.26%. In generale, l'utilizzo di un Learning Rate nell'intervallo $\{1e^{-5}, 2e^{-5}, 3e^{-5}\}$ produce risultati stabili e consistenti, con un valore di accuracy sempre superiore al 92% e un valore di F1-macro maggiore del 88%, confermando la robustezza del Full Fine-Tuning rispetto alla scelta degli iperparametri in questo intervallo.

Learning Rate	Batch Size	Accuracy	F1-Score
$2e^{-5}$	16	93.10%	89.26%
$1e^{-5}$	32	92.60%	88.80%
$2e^{-5}$	32	92.75%	88.40%
$3e^{-5}$	32	92.90%	88.28%
$3e^{-5}$	16	92.30%	88.16%
$1e^{-5}$	16	92.50%	88.03%

Tabella 7: Risultati delle configurazioni di Full Fine-Tuning

C. LoRA

La caratteristica fondamentale di LoRA è il numero di parametri addestrabili. Grazie a questa strategia anziché allenare il 100% dei parametri, ne sono stati allenati solamente 1,479,942 per le configurazioni con un rango di 16 e 2,364,678 per le configurazioni con rango pari a 32, un utilizzo pari al 1.8% dei parametri totali del modello. Per individuare la configurazione LoRA ottimale, è stata condotta una ricerca su 12 combinazioni di iperparametri tra Rango (16, 32), Learning Rate ($5e^{-5}$, $1e^{-4}$, $5e^{-4}$) e Batch Size (8, 16). La configurazione migliore ($r = 32$, LR $5e^{-4}$, BS 16) raggiunge un’accuracy del 92.65% e un F1-score macro dell’88.46%. In generale, $r = 32$ ottiene risultati leggermente superiori a $r = 16$ a parità di altri iperparametri. Il learning rate risulta essere l’iperparametro di maggior impatto. Valori bassi come $5e^{-5}$ producono performance inferiori, mentre valori di LR di $1e^{-4}$ e $5e^{-4}$ garantiscono risultati migliori. Si può osservare, però, un caso di *training collapse* nella configurazione ($r = 32$, LR $5e^{-4}$, BS 8), che crolla ad un’accuracy del 34.75% e un F1 dell’8.60%. Osservando la configurazione identica con BS 16 è possibile notare che non mostra questo problema, suggerendo che batch size ridotte possano amplificare l’instabilità del gradiente se combinati a valori di learning rate elevati e ranghi alti.

Rank	LR	BS	Accuracy	F1-Score
32	$1e^{-4}$	8	92.45%	88.61%
32	$5e^{-4}$	16	92.65%	88.46%
16	$5e^{-4}$	16	92.45%	88.46%
16	$5e^{-4}$	8	92.45%	88.36%
32	$1e^{-4}$	16	92.50%	88.24%
16	$1e^{-4}$	8	92.20%	88.08%
16	$1e^{-4}$	16	91.25%	86.87%
32	$5e^{-5}$	8	91.00%	86.60%
16	$5e^{-5}$	8	90.60%	86.42%
32	$5e^{-5}$	16	90.55%	86.41%
16	$5e^{-5}$	16	88.85%	84.15%
32	$5e^{-4}$	8	34.75%	8.60%

Tabella 8: Risultati delle configurazioni LoRA

D. In-Context Learning (Zero-Shot e Few-Shots)

È opportuno sottolineare che RoBERTa è un modello encoder-only addestrato con un obiettivo di Masked Language Modeling (MLM); a differenza dei modelli decoder generativi, non è stato progettato per l’ICL, e di conseguenza i risultati vanno interpretati

tenendo conto di questa considerazione. Con la configurazione zero-shot, il modello raggiunge la performance migliore, con un’accuracy del 51.15% e un F1-score macro del 26.73%. Il divario tra accuracy e F1-score macro suggerisce un comportamento sbilanciato del modello, che tende a predire correttamente le classi più frequenti ignorando quelle minoritarie. Con l’aggiunta di esempi nel contesto le performance peggiorano: la configurazione 1-shot registra il calo più marcato (accuracy 43.53%, F1 21.22%), mentre 3-shot e 5-shot mostrano un lieve recupero, pur rimanendo al di sotto della configurazione zero-shot. Tale andamento è prevedibile data l’architettura di RoBERTa-base. Il modello non è stato ottimizzato per il condizionamento del contesto come fa l’ICL e, di conseguenza, gli esempi aggiuntivi possono interferire con il pattern del prompt atteso dal MLM. È opportuno precisare che, in tutte le configurazioni, non sono stati registrati troncamenti del token <mask>, escludendo quindi risultati legati alla lunghezza del contesto.

Shots	Accuracy	F1-Score
0	51.15%	26.73%
1	43.53%	21.22%
3	45.77%	22.76%
5	46.35%	23.70%

Tabella 9: Risultati dell’In-Context Learning Zero-Shot e Few-Shots

3.5 Analisi degli errori

L’analisi degli errori, eseguita con il modello sul quale è stata applicata la strategia di Full Fine-Tuning con risultati migliori, mostra che il maggior numero di errori di classificazione si concentra su determinate classi di emozioni:

1. **Love e joy:** La classe *joy* risulta tra le più riconosciute dal modello. La classe *love*, invece, registra la performance peggiore con il 19% degli esempi classificati erroneamente come *joy*. Questo errore è semanticamente motivato: testi che esprimono amore condividono spesso un tono positivo e lessico affine a quelli di gioia, rendendo il confine tra le due classi ambiguo. Di seguito vengono mostrati alcuni esempi di errore:

- **Testo:** *‘i feel the need to work on caring’* → **True:** love, **Predicted:** joy
- **Testo:** *‘i was not feeling up to it yet i blamed my fiances deployment for bringing me down’* → **True:** love, **Predicted:** joy
- **Testo:** *‘i feel unimportant and small here lately’* → **True:** love, **Predicted:** joy

Su un totale di 2000 esempi, la classe *love* viene confusa con *joy* 30 volte.

2. **Surprise e Fear:** Anche in questo caso, la classe *surprise* risulta tra le più riconosciute, mentre *fear* è la classe che presenta il maggior numero di errori. Il 12% degli esempi di *fear* viene classificato come *surprise*, e il restante 8% si distribuisce tra *sadness* (4%) e *anger* (4%). La confusione con *surprise* è spiegabile dalla sovrapposizione contestuale delle due emozioni: entrambe implicano una risposta a eventi inattesi, e la distinzione tra timore e stupore può dipendere da sfumature lessicali sottili che il modello fatica a catturare. Di seguito vengono mostrati alcuni esempi di errore:

- **Testo:** *'i feel tortured'* → **True:** fear, **Predicted:** surprise
- **Testo:** *'i know that feeling awkward and not having friends in a space contributes to this'* → **True:** fear, **Predicted:** sadness
- **Testo:** *'im feeling less grumpy after that'* → **True:** fear, **Predicted:** sadness

Su un totale di 2000 esempi, la classe *fear* viene confusa con *surprise* 26 volte.

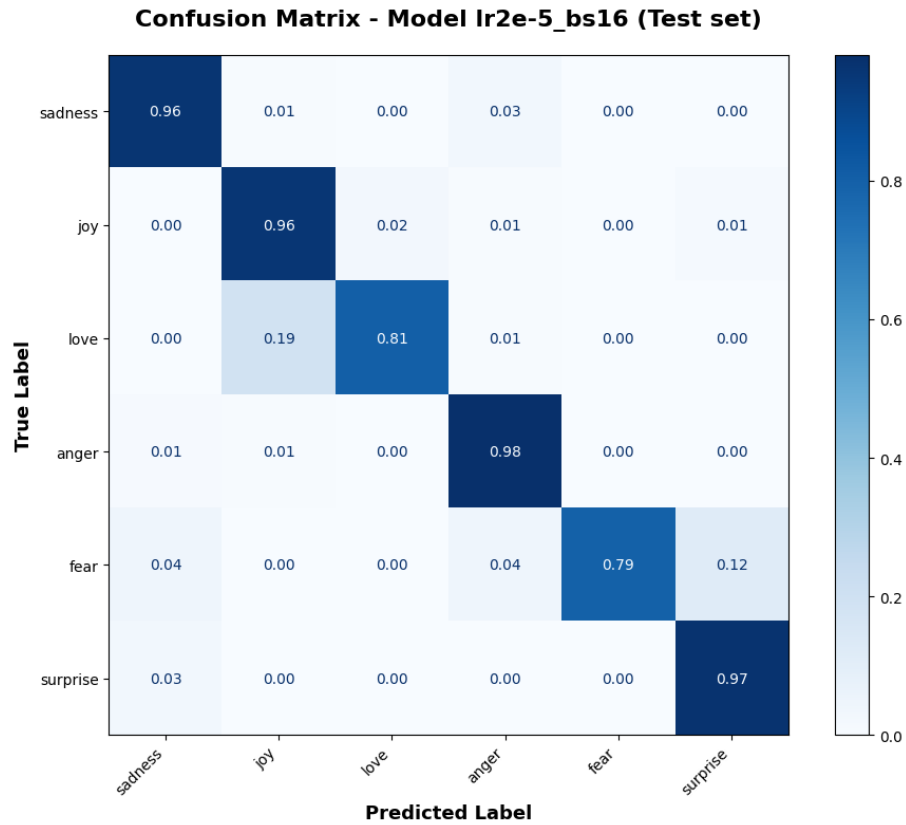


Figura 4: Matrice di Confusione del modello Full Fine-Tuned.

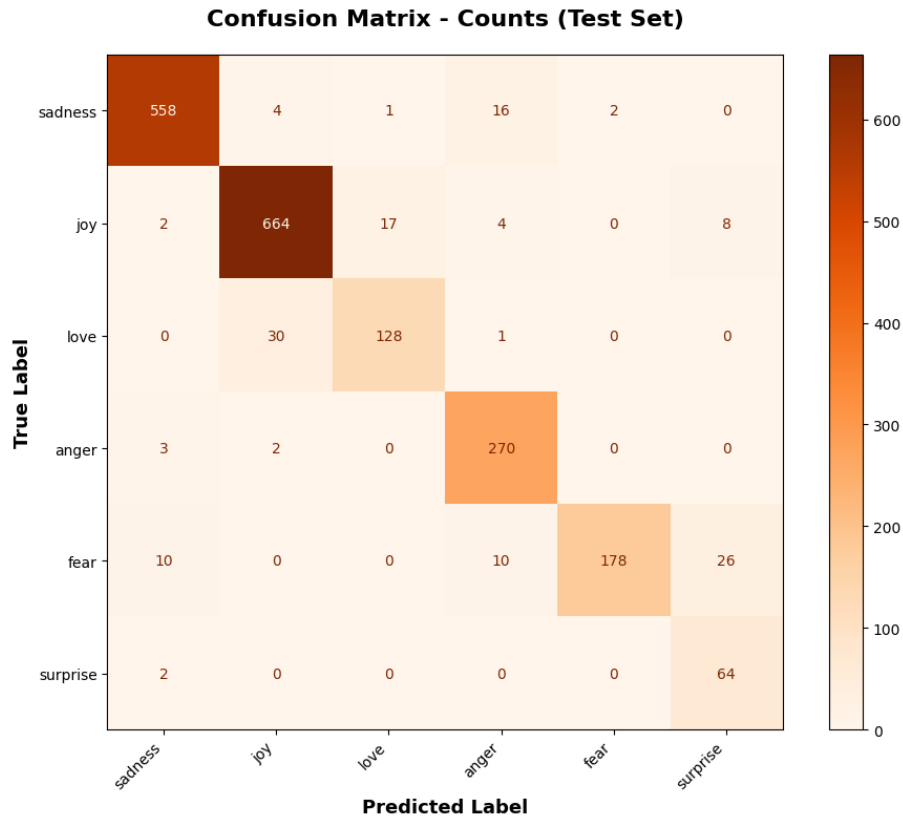


Figura 5: Matrice di Confusione Quantitativa del modello Full Fine-Tuned.

Di seguito vengono riportati gli errori più comuni con la relativa frequenza registrata:

Vera emozione	Predizione	Frequenza
love	joy	30 volte
fear	surprise	26 volte
joy	love	17 volte
sadness	anger	16 volte
fear	anger	10 volte
fear	sadness	10 volte
joy	surprise	8 volte
sadness	joy	4 volte
joy	anger	4 volte
anger	sadness	3 volte

Tabella 10: Frequenza degli errori più comuni registrati per RoBERTa-base

3.5.1 Risultati Finali e Confronto

Il seguente grafico riassume i risultati ottenuti dall'applicazione delle diverse strategie di training del modello RoBERTa-base. Si può subito notare come le strategie di Full Fine-Tuning e LoRA abbiano raggiunto risultati nettamente superiori rispetto alle altre strategie. Nel grafico è stata inserita anche la valutazione di RoBERTa-base con la testa di classificazione casuale che a causa dello sbilanciamento delle classi raggiunge un accuracy del 29.05% e un F1 macro del 7.50%, rispetto a un accuracy del 16% che si otterrebbe con un dataset bilanciato.

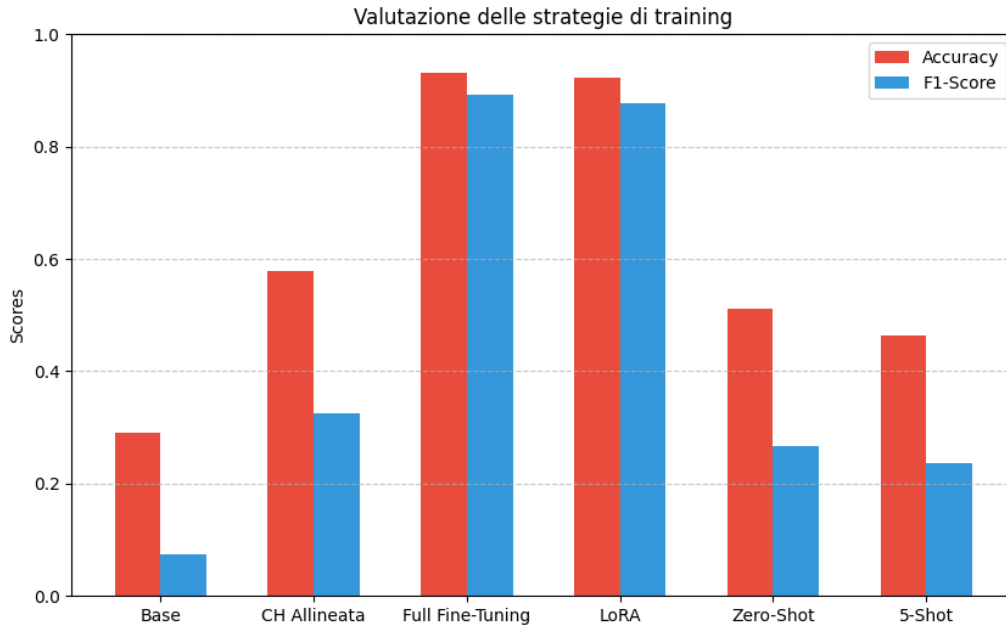


Figura 6: Confronto di Accuracy e F1 macro ottenuti tra le diverse strategie testate.

Per concludere, di seguito si può trovare una tabella che riassume i migliori risultati per ciascuna strategia utilizzata:

Strategia	Accuracy	F1	Conclusioni
Transfer Learning	57.95%	32.50%	Performance limitate. Le predizioni si concentrano sulle classi più frequenti.
Full Fine-Tuning	93.10%	89.26%	Miglior strategia. L'aggiornamento di tutti i pesi consente al modello di specializzarsi sul task.
LoRA	92.35%	87.77%	Ottimo compromesso tra efficienza e performance. Ottimo per risorse limitate
ICL (0-Shot)	51.15%	26.73%	Strategia non adatta a modelli encoder-only.

Tabella 11: Riepilogo finale Ablation Study

3.5.2 Results Discussion

Dai risultati ottenuti è facile determinare che, per il task di text emotion classification utilizzando un modello encoder-only come RoBERTa-base, le due strategie più efficienti sono il **Full Fine-Tuning** e **LoRA**. Il Full Fine-Tuning risulta la strategia che ha ottenuto risultati migliori, ma con un costo computazionale maggiore dovuto alla richiesta di un elevato numero di risorse per l'aggiornamento del 100% dei parametri. In termini di risorse disponibili, LoRA risulta altamente più efficiente rispetto al Full Fine-Tuning, con una perdita del 0.75% sul valore di accuracy e del 1.49% sul valore di F1 macro. I risultati ottenuti sono più che soddisfacenti, in particolare visto lo sbilanciamento delle classi del dataset; con un valore di F1 macro del 89.26% si è dimostrato che il modello risulta in grado di differenziare efficacemente anche le classi meno frequenti come **surprise** e **love**.

3.5.3 Method Validity

I risultati ottenuti con la strategia di Full Fine-Tuning di RoBERTa-base confermano la validità dell’approccio adottato per il task di *text emotion classification*. Al tempo stesso una strategia come l’ICL mostra come non sia adatta a un modello encoder-only per questo tipo di task.

3.5.4 Limitations and Maturity

Il lavoro presenta alcune limitazioni che è opportuno considerare nell’interpretazione dei risultati. Il dataset utilizzato è sbilanciato, di conseguenza, classi come *love* e *fear* sono molto frequenti rispetto ad altre, il che si riflette nel divario tra accuracy e F1-score macro osservato nelle diverse strategie testate. Inoltre, l’utilizzo della strategia di ICL, su un modello non adatto, ha fornito risultati che non sono comparabili per un discorso di efficienza rispetto a modelli decoder generativi come GPT, per i quali l’ICL è stato progettato.

3.5.5 Future Works

Il seguente lavoro potrebbe essere esteso con diversi sviluppi futuri. Un primo contributo significativo sarebbe l’estensione dell’analisi a modelli decoder generativi, come LLaMA o GPT, per una valutazione comparativa dell’efficienza dell’ICL in un contesto architetturale più adeguato. Questo consentirebbe di valutare concretamente il divario di performance rispetto all’approccio encoder-only utilizzato. Sarebbe, inoltre, interessante estendere l’analisi a modelli multilingua, come XLM-RoBERTa, per valutare lo stesso approccio su testi in lingue diverse dall’inglese e ampliare il campo di applicazione del sistema sviluppato.

3.6 Limitations and Maturity

- **Limiti:** La *Context Window* limitata impedisce l’uso di prompt complessi o catene di ragionamento. Il dataset presenta un sbilanciamento naturale che penalizza leggermente le classi minoritarie (*Surprise*).
- **Maturità:** La soluzione sviluppata, basata su librerie standard industriali (*transformers*, *peft*), ha raggiunto un buon livello di maturità tecnologica, pronta per essere integrata in pipeline di produzione con costi di inferenza contenuti.

3.7 Future Works

Per avanzare ulteriormente nel progetto e superare i limiti identificati, proponiamo le seguenti direzioni di ricerca:

1. **Data Augmentation via Back-Translation:** Per mitigare lo sbilanciamento delle classi (in particolare *Surprise*, che ha pochi campioni), proponiamo di generare dati sintetici utilizzando una pipeline di traduzione andata-ritorno (es. Inglese → Francese → Inglese). Questa tecnica introduce variazioni lessicali e sintattiche naturali mantenendo inalterata la semantica originale, arricchendo il training set con esempi diversificati che rendono il modello più robusto sulle classi minoritarie.

2. **Supervised Contrastive Learning:** Per risolvere l'ambiguità tra classi semanticamente sovrapposte come *Love* e *Joy*, suggeriamo di introdurre una fase preliminare di addestramento con *SupCon Loss*. A differenza della classica Cross-Entropy (che valuta i campioni singolarmente), la loss contrastiva lavora su coppie: forza l'encoder ad avvicinare drasticamente le rappresentazioni vettoriali dei tweet della stessa classe e ad allontanare quelle di classi diverse nello spazio latente. Questo creerebbe confini decisionali più netti prima ancora di addestrare il classificatore finale.
3. **Task-Adaptive Pre-Training:** Invece di passare direttamente dal modello generico (BERTweet) al fine-tuning, proponiamo una fase intermedia di "riscaldamento". Questa consiste nel continuare il pre-training con obiettivo *Masked Language Modeling* (MLM) utilizzando i soli testi del dataset **emotion** (senza etichette). Questo adatta i pesi dell'encoder allo specifico vocabolario, slang e distribuzione linguistica del nostro dataset, ottimizzando la capacità del modello di estrarre feature rilevanti da contesti emotivi ambigui prima di applicare LoRA.

References

- [1] E. Saravia, H.-C. T. Liu, Y.-H. Huang, J. Wu e Y.-S. Chen, “CARER: Contextualized Affect Representations for Emotion Recognition,” in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, Brussels, Belgium: Association for Computational Linguistics, ott. 2018, pp. 3687–3697. DOI: [10.18653/v1/D18-1404](https://doi.org/10.18653/v1/D18-1404) indirizzo: <https://www.aclweb.org/anthology/D18-1404>
- [2] D. Q. Nguyen, T. Vu e A. Tuan Nguyen, “BERTweet: A pre-trained language model for English Tweets,” in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, Q. Liu e D. Schlangen, cur., Online: Association for Computational Linguistics, ott. 2020, pp. 9–14. DOI: [10.18653/v1/2020.emnlp-demos.2](https://doi.org/10.18653/v1/2020.emnlp-demos.2) indirizzo: <https://aclanthology.org/2020.emnlp-demos.2/>
- [3] E. J. Hu et al., “LoRA: Low-Rank Adaptation of Large Language Models,” *CoRR*, vol. abs/2106.09685, 2021. arXiv: [2106.09685](https://arxiv.org/abs/2106.09685). indirizzo: <https://arxiv.org/abs/2106.09685>
- [4] Y. Liu, N. G. Myle Ott, M. J. Jingfei Du, O. L. Danqi Chen, L. Z. Mike Lewis e V. Stoyanov, “RoBERTa: A Robustly Optimized BERT Pretraining Approach,” in *RoBERTa: A Robustly Optimized BERT Pretraining Approach*, Q. Liu e D. Schlangen, cur., Online: Association for Computational Linguistics, lug. 2019, pp. 1–13. DOI: [10.48550/arXiv.1907.11692](https://doi.org/10.48550/arXiv.1907.11692) indirizzo: <https://arxiv.org/pdf/1907.11692>